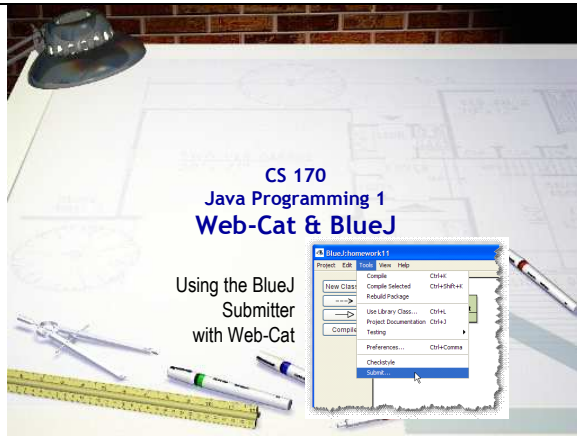




Slide 1
CS 170
Java Programming 1
Web-Cat & BlueJ
 Duration: 00:01:09
 Advance mode: Auto

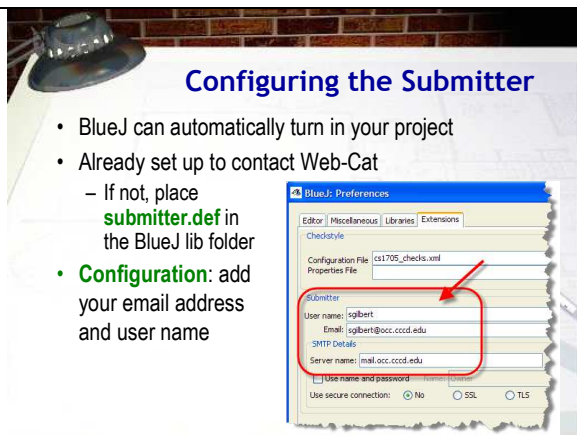
Notes:

Hi there. This is the CS 170, Java Programming 1 lecture, Web-Cat & BlueJ.

You first met Web-Cat back in the second week of class. Up to now, though, you've used the Web interface to upload your files. That works OK when you have only a single file; when you have multiple files in an assignment, though, you need to package them correctly in a zip file, which is easy to do incorrectly.

Instead of manually zipping up the necessary files in your project, we can use a BlueJ feature, called the Submitter, which will locate the required files for each assignment, archive them into a single zip file, and upload them to the Web-Cat server. I've already done the initial submitter configuration when I customized the OCC version of BlueJ. You'll still need to customize the submitter for your own account though.

In this lecture, you'll learn how to configure the BlueJ submitter and use it to turn in your multi-file programming assignments to Web-Cat. We'll also take another look at some of the Web-Cat reports.



Slide 2
Configuring the Submitter
 Duration: 00:01:29
 Advance mode: Auto

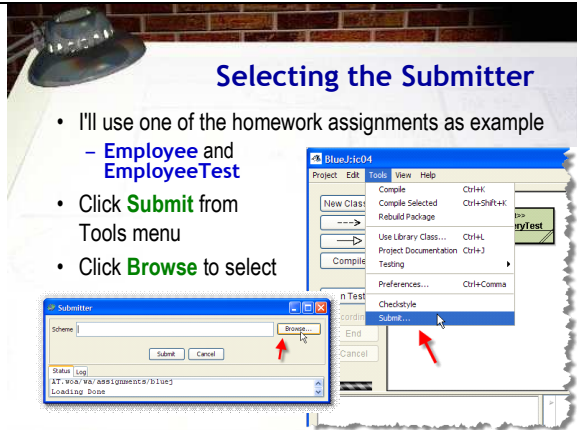
Notes:

The BlueJ submitter plug-in is an add-on program for BlueJ that allows you to bundle and submit your homework assignments automatically. The submitter should already be configured to contact Web-Cat when you use the Submit button. If it isn't, (if, for instance, you're using a different version of BlueJ than the one we're using in the lab), simply grab a copy of the file submitter.def from this week's class Web page and place it in BlueJ's lib folder on your local machine.

To activate the submitter, though, you still need to configure it for your personal account. You should only need to do this once for your copy of BlueJ. In the lab, your configuration should travel around with you. To configure the submitter

- open the Preferences dialog from the Tools menu.
- choose the Extensions tab
- fill in the username with your Web-cat user name. This should be the same as your OCC email name without the @student.occ.cccd.edu part.
- fill in your email address
- Use mail.occ.cccd.edu for the mail server. We don't actually use this to submit to Web-Cat, so it doesn't make any difference what you put here.

Click the OK button to dismiss the Preferences dialog.



Slide 3

Selecting the Submitter

Duration: 00:00:49

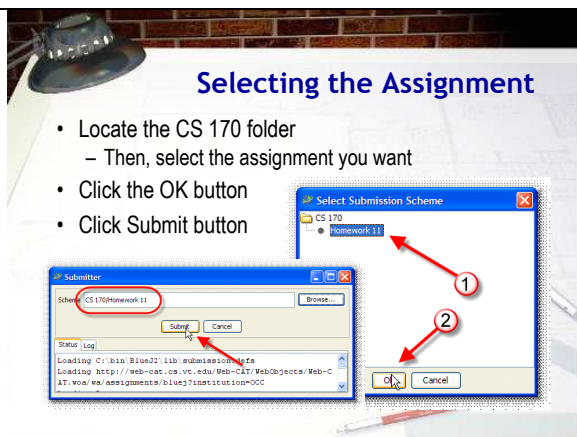
Advance mode: Auto

Notes:

To show you how to use the submitter, I'll use a homework assignment from the textbook: Exercise P3.4, the Employee and EmployeeTest programs. If you weren't assigned this problem this semester, you were assigned a similar problem from the end of Chapter 3.

After you've defined your class, completed the documentation and written your JUnit tests, choose Submit from the Tools menu to upload your program.

When the Submitter dialog appears, click the Browse button to select the assignment you want to upload. (If you're submitting a second time, the Scheme text area will already be filled in with the last selection you made. Double check the entry there to make sure you don't try submitting the wrong assignment.)



Slide 4

Selecting the Assignment

Duration: 00:00:42

Advance mode: Auto

Notes:

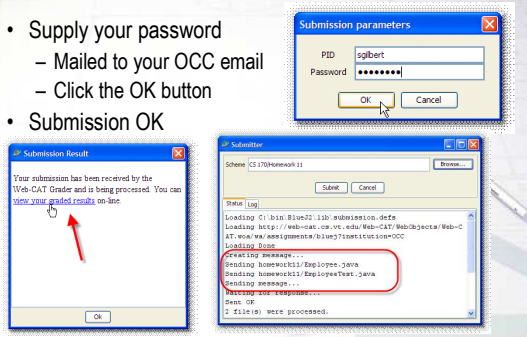
When the Select Submission dialog appears, locate the folder for CS 170 and expand it. Inside, you'll find all of the submissions that are still "open". In my picture here, only Homework 11 is active, but you'll normally find two, three or more assignments active in any given week. Make sure that you select the assignment you're currently working on.

When you've done that, click the OK button.

When you do, the Submitter dialog reappears, but this time, the homework assignment you selected has been entered into the Scheme text area at the top of the dialog. Now click the Submit button.

Log In and View Results

- Supply your password
 - Mailed to your OCC email
 - Click the OK button
- Submission OK



Slide 5

Log In and View Results

Duration: 00:01:31

Advance mode: Auto

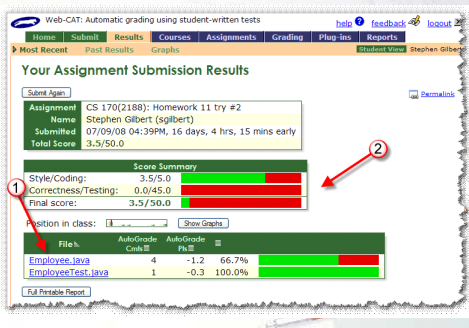
Notes:

When you click the Submit button, you'll be asked to supply your password. Your username should be filled in from the Preferences dialog you completed on an earlier slide. You'll find your Web-Cat password in your OCC email account. You can change the password if you like. (This is the same password you used to log-in to Web-Cat using your Browser.) Click OK to upload your files.

If your submission was successful, you'll see a dialog with a hyperlink. You can click this to launch the default browser already logged into your Web-Cat site. If the default browser is different than the one you usually use (like it is in the lab for me), you can open the Web-Cat site in your normal browser and just close the dialog shown here. If you supplied the wrong password, or, if you aren't registered for the class, you'll see a different dialog with an error message.

When you submit your assignment, the submission definition stored on the server determines what files are sent to the server. For Homework 11, shown in the slide here, only two files are submitted: Employee.java and EmployeeTest.java, even if other files exist in the project. That's also why you need to make sure you follow the homework instructions precisely; if you use another name for your test program, TestEmployee, for instance, it would be uploaded when you submitted the assignment.

Overall Results



Slide 6

Overall Results

Duration: 00:01:05

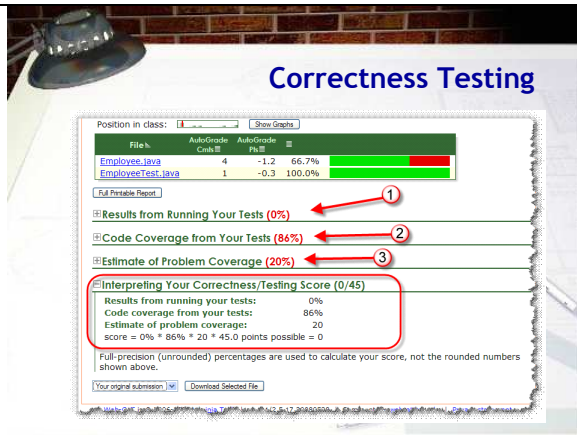
Advance mode: Auto

Notes:

Now let's go over the results a little more. For this lecture, I quickly created the Employee class, added a constructor and two accessors, and wrote a test program with a single test which failed. When I uploaded the assignment, this was the result.

- You can see that both of my files were uploaded. Clicking the hyperlink shows you the source code. To the right of each of the files you'll see some red and green bars. Those bars show you how much of your program was tested by your own unit tests. You can see that only 66.7% of Employee.java has actually been tested. You'll also notice to the right of the file name the number of comments and the number of style points that were deducted for each file.
- Above that you'll notice the summary for this assignment. I got 3.5 out of 5 style points but 0 out of 45 correctness or testing points.

Let's scroll down and see how the testing was scored.



Slide 7

Correctness Testing

Duration: 00:01:35

Advance mode: Auto

Notes:

Your correctness score is a combination of three factors:

- First is the percentage of your unit tests that pass. You should make sure that all of your unit tests pass before submitting your assignment, of course. Here, I had only a single test and it failed, so I got 0% for this portion.
- Second is the percentage of your code that is covered by your tests. In my example here, I only tested 86% of my program. Even if all of my tests had passed, I still wouldn't have tested all of the methods in my code. (At this point, I'm only measuring the number of methods that you've tested. At the end of the course I'll switch to measuring the number of statements, so you can be sure that every line of code is tested.)
- Finally, there's the estimate of problem coverage, which is 20% in this case. After I run the tests you supply, I run a second set of tests that I've written. These reference tests are designed to make sure that your code meets all of the problem specifications.

Your final score for the correctness portion of the program is calculated by multiplying all three of these percentages together. Of course, if one of the percentages is 0, as it is in this case, the end result is 0 no matter how well you did on the other three.

Let me go ahead and fix some of the code in my project and submit again. Then we'll take a closer look at code coverage and reference hinting.



Slide 8

Coverage, Style and Hints

Duration: 00:01:27

Advance mode: Auto

Notes:

OK, that looks much better. As you can see, I've only got one problem in the style section, and I've got 36 of 45 points for the correctness score. Let's see how those were calculated.

When I scroll down you can see that now all of my tests pass and I have 100% code coverage; that is, I'm testing 100% of my own code with 100% success. The only problem is that my class only meets 80% of the program requirements. As I mentioned, this is determined by using a set of reference tests.

Unfortunately, you don't get to see the results from failed reference tests--instead, only the results from your own tests are shown. That can be frustrating to get absolutely no feedback about how or why your work is inconsistent with the problem.

To remedy this, Web-Cat provides "hints", as you can see at the bottom of this page, that will provide you with a glimpse into why your submission was downgraded. In this case, I've forgotten to supply one of the methods required by the problem specification.

To see why you were marked down for style, just click the

	hyperlinked name of your file. In this case, you can see that I forgot to end the first sentence of a JavaDoc comment with a period, like the style-guide requires.
--	---